

LangGraph Edges

Technical Guide to Graph Routing and Execution Flow

In LangGraph, edges define the execution flow between nodes. While nodes perform the computation and manipulate the state, edges determine which node executes next based on standard sequential flow or dynamic conditional logic.

Literal Execution Context

Edges do not alter the graph state. They exclusively read the state (in the case of conditional edges) to execute routing functions that return the string identifier of the next node to invoke.

1. Standard Edges (add_edge)

A standard edge creates a deterministic, unconditional path from a source node to a destination node. When the source node finishes execution, the destination node is immediately invoked.

```
from langgraph.graph import StateGraph

# Assuming 'workflow' is a StateGraph and nodes "node_a" and "node_b" exist
# workflow.add_node("node_a", function_a)
# workflow.add_node("node_b", function_b)

# Literal execution: Add a standard edge
workflow.add_edge("node_a", "node_b")
```

2. Special Entry and Exit Edges

LangGraph uses `START` and `END` virtual nodes to manage graph boundaries. Standard edges are used to connect these virtual nodes to your functional nodes.

```
from langgraph.graph import START, END

# Define where the graph execution begins
workflow.add_edge(START, "initialize_node")

# Define where the graph execution terminates
workflow.add_edge("final_output_node", END)
```

3. Conditional Edges (add_conditional_edges)

Conditional edges evaluate the current state via a routing function to determine the next node dynamically. The routing function must return a string (or list of strings for parallel execution) matching the name of registered nodes.

Defining the Routing Function

The routing function receives the current graph `State` as its argument and performs logic to decide the next step.

```
def router_function(state: AgentState) -> str:
    # Literal execution: evaluate state keys
    messages = state.get("messages", [])
    last_message = messages[-1]

    # Check if the last AIMessage contains tool calls
    if hasattr(last_message, 'tool_calls') and last_message.tool_calls:
        return "tools"

    # Otherwise, end execution
    return "end"
```

Registering the Conditional Edge

You attach the conditional edge to a source node. You provide the routing function, and optionally, a dictionary mapping the function's string outputs to the actual node names in the graph (this mapping is often omitted if the function directly returns valid node names like "tools" or `END`).

```
# Mapping router output to graph nodes
workflow.add_conditional_edges(
    "agent",          # Source node
    router_function,  # Routing function to execute
    {
        "tools": "tools_node", # If router returns "tools", route to "tools_node"
        "end": END             # If router returns "end", route to END
    }
)
```

4. Advanced: Path Mappings and Direct Returns

If your routing function directly returns the exact names of the target nodes (or `END`), you can omit the path mapping dictionary entirely. LangGraph will automatically resolve the returned string to the node identifier.

```
def direct_router(state: AgentState) -> str:
    if state["error"]:
        return "error_handler_node"
    return END

# Literal execution without mapping dictionary
workflow.add_conditional_edges(
    "process_data_node",
    direct_router
)
```